

Planejamento e Escopo com IA

Requirements Copilot (PT-3)

Ferramentas de IA para Gestão de Projetos · Módulo 1.3

Prof. Ahirton Lopes, Ph.D.

4 Tipos de Alucinação

01

Especificação inventada

O stakeholder disse "em tempo real". O modelo escreveu "latência < 200ms". Esse número não veio do stakeholder - veio do padrão estatístico do treinamento. Sem o protocolo de ambiguidade, esse SLA vai para o backlog como compromisso.

02

Dependência não mapeada

"Exportar relatório para o sistema de RH" - parece razoável. Mas o modelo não verificou se o RH tem API, qual o formato, qual o SLA de disponibilidade. A integração entra no sprint e ninguém mapeou a dependência técnica.

03

Viabilidade técnica silenciosa

"Detecção de comportamento anômalo por machine learning" - elegante. Mas o cliente tem infraestrutura? Dado histórico suficiente? Alguém no time para manter? O Requirements Copilot não sabe - e não deve saber. Essa decisão é do arquiteto.

04

Gold plating silencioso

O Carlos pediu alerta de velocidade. O modelo gerou três limiares configuráveis, dashboard de tendências e integração com telemetria. Tecnicamente bonito - mas o Carlos não pediu nada disso. Cada linha do critério de aceite precisa ter correspondência direta com algo que o stakeholder disse.

Checklist de Curadoria

Antes de aprovar qualquer output:

1

O requisito é tecnicamente viável na arquitetura atual?

Não no que poderia ser construído - no que existe hoje ou no que o projeto tem orçamento para construir.

2

Existe exposição de compliance ou LGPD?

Qualquer requisito que envolva dados pessoais, localização ou comportamento de usuário precisa dessa checagem antes de entrar em desenvolvimento.

3

Esse requisito cria dependências não mapeadas?

Com outros módulos ou sistemas externos. Se sim, a história não entra no sprint antes de ter as dependências mapeadas.

O que o modelo **não sabe**

Essas informações não estão na transcrição - estão na sua cabeça.

1

Limitações de arquitetura interna

O servidor legado tem limitação de payload que pode invalidar um requisito inteiro. O stakeholder não mencionou - está na sua cabeça.

2

SLA do integrador externo

O sistema de RH tem SLA de 6 semanas. A User Story de integração não entra no sprint sem isso mapeado.

3

Contexto de auditoria e compliance

O cliente acabou de passar por uma auditoria de LGPD. Qualquer requisito com dados pessoais precisa dessa checagem antes de entrar em sprint.

4

Rate limiting de sistemas externos

A API de rastreamento aceita 100 chamadas/min. Um requisito de alertas em tempo real para 140 motoristas simultâneos invalida essa arquitetura - antes mesmo de chegar ao sprint.

5

Restrições contratuais com fornecedores

O contrato com a montadora proíbe modificações no firmware dos veículos. Isso elimina categorias inteiras de soluções que o modelo pode sugerir como tecnicamente elegantes.

Missão Prática #01

01

Configure e rode o pipeline

Cole o system prompt no Gemini ou no modelo que você usa. Use a transcrição do Carlos nos materiais. Documente o output completo.

02

Identifique onde o modelo errou

Onde ele inventou especificação?
Onde marcou ambiguidade que estava clara? Onde faltou flag de risco que você como dev sênior teria visto? Documente ≥ 2 pontos.

03

Ajuste o prompt e rode novamente

O que você mudou? Por quê? O segundo output foi melhor? Em que medida? Entrega: relato técnico de uma iteração de prompt engineering - não três User Stories prontas.

“Requisitos estruturados.
Sem priorização baseada em dados,
é um backlog de opinião.”

- Ahirton Lopes

Módulo 2 →

Priorização Inteligente de Backlog

*Por que estimativa pontual é sistematicamente errada - e como o RICE Score
e o WSJF mudam a conversa com o negócio.*